

Calculadora com Jest, ESLint e GitHub Actions

Arquivo	Função
calculator.js	Soma, subtração, multiplicação, divisão
calculator.test.js	8 testes unitários Jest
.eslintrc.json	Regras ESLint (no-var, eqeqeq, semi...)
.github/workflows/ci.yml	Pipeline: lint → test no push para main
codigo_ruim.js	Código ruim intencional (Etapa 4)
codigo_melhorado.js	Código revisado pela IA

Etapa 0 — Setup do Repositório

Crie o repositório no GitHub (github.com → New repository → nome: **calculadora-jest**) e depois rode os comandos abaixo dentro da pasta do projeto.

```
git init
git add .
git commit -m "feat: projeto calculadora com Jest"
git remote add origin https://github.com/SEU_USUARIO/calculadora-jest.git
git branch -M main
git push -u origin main
```

■ *PRINT AQUI: repositório no GitHub com os arquivos visíveis.*

Etapa 1 — Testes Passando ■

Instale as dependências e rode os testes. Todos os 8 devem passar.

```
npm install
npm test
```

Saída esperada:

```
PASS ./calculator.test.js
  ✓ soma 2 + 3 = 5
  ✓ soma negativo -1 + 1
  ✓ subtrai 10 - 4 = 6
  ✓ subtrai com negativo
  ✓ multiplica 3 * 4 = 12
  ✓ multiplica por zero
  ✓ divide 10 / 2 = 5
  ✓ divide por zero lança erro
```

Tests: 8 passed, 8 total ■

■ *PRINT AQUI: terminal com os 8 testes verdes.*

Commit:

```
git add .
git commit -m "feat: calculadora e testes Jest"
git push
```

■ *PRINT AQUI: confirmação do git push no terminal.*

Etapa 2 — Quebrando Teste de Propósito ■

Edite `calculator.test.js` e troque o valor esperado de um teste:

```
// ANTES (passa):  
expect(add(2, 3)).toBe(5);  
  
// DEPOIS (quebra de propósito):  
expect(add(2, 3)).toBe(99);  
  
npm test
```

Saída esperada (falha):

```
FAIL ./calculator.test.js  
  ✕ soma 2 + 3 = 5  
  
    Expected: 99  
    Received: 5
```

```
Tests: 1 failed, 7 passed ■
```

■ **PRINT AQUI:** *terminal com o FAIL vermelho.*

Reverter depois do print:

```
# Desfaz a alteração  
git checkout -- calculator.test.js  
npm test    # volta ao verde
```

Etapa 3 — GitHub Actions: Verde e Vermelho

O arquivo `.github/workflows/ci.yml` define o pipeline que roda automaticamente em cada push para main.

Pipeline:

```
on: push → main

Jobs (em sequência):
  1. npm run lint    (ESLint)
  2. npm test        (Jest)
```

3a — Pipeline verde ■

Faça o push com tudo correto:

```
git add .github/
git commit -m "ci: adiciona GitHub Actions"
git push
```

Acesse: GitHub → seu repositório → aba **Actions**.

■ *PRINT AQUI: aba Actions com os dois jobs verdes (lint + test).*

3b — ESLint falhando ■

Crie o arquivo `eslint_ruim.js` com código que viola as regras:

```
// eslint_ruim.js
var x = 1
if(x == 2) console.log("erro")

git add eslint_ruim.js
git commit -m "test: ESLint falhando de propósito"
git push
```

■ *PRINT AQUI: aba Actions com o job ESLint vermelho.*

Restaurar (remover o arquivo ruim):

```
del eslint_ruim.js          # Windows
# ou
rm eslint_ruim.js          # Mac/Linux

git add eslint_ruim.js
git commit -m "fix: remove arquivo ESLint ruim"
git push
```

■ *PRINT AQUI: aba Actions voltando ao verde.*

Etapa 4 — Código Ruim → IA → Código Melhorado

Commit 1 — Código ruim

```
git add codigo_ruim.js
git commit -m "feat: codigo_ruim antes da revisao"
git push
```

■ *PRINT AQUI: GitHub mostrando o arquivo codigo_ruim.js.*

Prompt para a IA (ChatGPT / Copilot / Cursor)

```
Revise este código como um Senior Software Engineer.
Analise: segurança, clean code, bugs, legibilidade,
performance, boas práticas e problemas futuros.
```

```
[colar o conteúdo de codigo_ruim.js]
```

■ *PRINT AQUI: tela da IA com o prompt e a resposta.*

Problemas que a IA vai apontar:

- var ao invés de const/let → no-var violado
- Operador == ao invés de === → comparação frouxa
- Divisão por zero não tratada → retorna Infinity silenciosamente
- Parâmetro string mágica ('soma', 'sub') → difícil de manter
- Nenhuma validação de tipo nos argumentos
- Sem semicolons, sem JSDoc

Commit 2 — Código melhorado

```
git add codigo_melhorado.js
git commit -m "refactor: melhorias sugeridas pela IA"
git push
```

■ *PRINT AQUI: diff do GitHub mostrando ruim vs melhorado lado a lado.*

Checklist de Entrega

- Repositório GitHub criado com os arquivos
- Print: 8 testes passando (npm test verde)
- Print: teste falhando de propósito (npm test vermelho)
- Print: Actions verde (lint + test OK)
- Print: Actions vermelho (ESLint falhando)
- Print: Actions voltando ao verde
- Print: commit codigo_ruim.js no GitHub
- Print: prompt + resposta da IA
- Print: commit codigo_melhorado.js no GitHub

Todos os arquivos estão prontos. Suba a pasta para o GitHub e siga o guia acima.